EE 685 Assignment: A SIMD MIPS

Coleman P. Earlywine
University of Kentucky
Department of Electrical and Computer Engineering
Lexington, Kentucky, USA
coleman.earlywine@uky.edu

Stefan Hustrulid
University of Kentucky
Department of Electrical and Computer Engineering
Lexington, Kentucky, USA
stefan.hustrulid@uky.edu

David R. Stewart II
University of Kentucky
Department of Electrical and Computer Engineering
Lexington, Kentucky, USA
david.stewart@uky.edu

Abstract

This project turns the course's five-stage pipelined MIPS into a SIMD machine. The control unit (CU) fetches and decodes every instruction, owns the memory, and runs lw, sw, beq and the MIPS arithmetic. `‘PES` processing elements (PEs) each have 32 registers and an ALU and run the broadcast PE instructions on their own registers. All instructions in the AIK specification are implemented, and the design was tested with (`‘PES = 2, 4, 8`).

1 General Approach

We divided the work into three parts. Part 1 (Coleman Earlywine) covers the CU, instruction decode, the pipeline registers, branches, hazards, and forwarding. Part 2 (Stefan Hustrulid) covers the PE registers, PE arithmetic, the enable logic, and PE writeback. Part 3 (David Stewart) covers the communication instructions `gor`, `bcast`, and `net`, along with the team test program.

1.1 Pipeline Organization

The pipeline has the five stages fetch, decode, execute, memory, and writeback, with several instructions in different stages at once. The buffers between stages are named for the stage that fills them: the `ID_*` buffers hold decode's outputs and feed execute, the `EX_*` buffers feed memory, and the `MEM_*` buffers feed writeback. Each buffer holds CU values and, alongside them, PE values with one 32-bit slice per PE, where PE p uses bits `[p*32 +: 32]`. CU register 5 and PE register 5 are separate locations.

Each instruction carries a flag saying whether its result goes to a CU or a PE register, and the decoder works out separately where its inputs come from. This is what lets `gor` read the PEs but write the CU, and `bcast` read the CU but write the PEs. Opcode 0 selects CU register instructions and opcode 16 selects PE register and special instructions. A PE immediate opcode is its CU opcode plus 16, so the decoder subtracts 16 and reuses the CU cases. The ALU control gains a fifth bit that marks a PE destination while the other four bits are the example's ALU codes. The six special instructions are numbered 1 `offeq`, 2 `pushen`, 3 `popen`, 4 `gor`, 5 `bcast`, and 6 `net`. An instruction that writes no register is given destination 0, and PE writes to registers 28-31 are blocked the same way.

The CU ALU is combinational, and the PE results must also be ready before the clock edge. Each piece of state has a single writer, which avoids ordering problems between clocked blocks.

1.2 Forwarding

Decode may need a value that an older instruction has not yet written back. A match requires a real instruction, a nonzero destination, the same register number, and the same bank (CU or PE), so a CU register 5 result is never forwarded into a PE register 5 read or vice versa. For CU values, the newest result wins in the order of execute, memory, then writeback. A load's data is not ready in execute, so an instruction that needs it waits one cycle. Each PE has its own pair of forwarding paths, built with a generate loop. If an older instruction was disabled in a PE, it will never write that PE's register, so that PE skips it and uses an older value.

There are three reasons to stall: a needed value from a load, PE support not being ready, and a pending `offeq`, `pushen`, or `popen`. For the last case, later SIMD instructions wait until the new enable state is written. During a stall, fetch holds its instruction and decode sends an empty bubble down the pipeline.

1.3 CU Memory, Branches, and Halting

Instructions and CU data share one memory of 512 32-bit words, and a byte address is shifted right by two to find the word. PEs have no load or store access. A `beq` compares its forwarded values in decode. When the branch is taken, the next sequential instruction is skipped and the target is fetched instead. There is no branch delay slot. An unsupported instruction, such as an all-zero word, stops further fetching and halts the processor when it reaches writeback.

1.4 PE State and Enable Logic

The PE registers are stored as one array

```
reg [31:0] PE_reg[0:31][0:`PES-1],
```

with 32 registers of 32 bits for each PE, where the first index is the register and the second is the PE. On reset, registers 0-28 are set to 0, `$iproc` (29) is set to the PE number, `$nproc` (30) to `‘PES`, and `$live` (31) to 1, since `$dl` (28) starts at 0.

In the execute stage, each PE's ALU takes its operation from the saved ALU control and its inputs from its own slices of the saved operands. The results are packed into one wire that carries them to the next stage. The special instructions are identified by their saved number. For `offeq`, `pushen`, and `popen`, each PE computes a proposed new `$dl`:

- `offeq`: keep `$dl` if it is nonzero; otherwise set it to 1 if the two registers are equal
- `pushen`: add 1 if `$dl` is nonzero
- `popen`: subtract 1 if `$dl` is nonzero.

The proposed values move through the execute and memory buffers along with their instruction. When the instruction reaches writeback, `$dl` and `$live` are updated from them in every PE, including disabled ones. Ordinary PE results are written only in PEs that were enabled when the instruction was decoded, and never

to the special registers. Coleman added the reset clearing of registers 0-28, the result buffer and its connection to the pipeline, the proposed `$dl` signals, and the final `$dl`/`$live` update.

1.5 Communication

Part 3 only computes the three results. Part 1 already decodes these instructions, forwards their operands, and routes their results, and Part 2's writeback stores them, so no changes to Parts 1 or 2 were needed.

As the assignment specifies, `gor` ORs one register across all PEs, enabled or not, and puts the result in a CU register, so `gor $live, $x` gives 0 when no PE is live. We wrote it as a chain of OR steps built with a generate loop, because the assignment recommends generate loops for per-PE hardware, it matches the example program's wire style, and it has the same structure as the course's parity generator. The `bcast` instruction copies a CU value into every PE. In `net`, each PE uses its own register value as an index to pick one PE's copy of another register (multiplexer). Both `bcast` and `net` write only enabled PEs, which Part 2's writeback enforces.

1.6 Instruction Encoding

We used the version of the AIK specification that lists one instruction per line as it was easier to read and edit. The operand order follows the AIK specification, so `gor` and `bcast` name the source first. AIK prints one byte per line, with most significant byte first, and every four bytes forming one memory word.

1.7 Testing

The test program runs each new instruction once with results checked via the trace. The testbench's clock and reset loop is the one from the example code provided in the assignment specification. It first fills memory with zeros to make it so any unused word is an illegal instruction and then uses an all zero word at the end to halt. The values are chosen so each result is easy to check:

- CU registers start at 0, so `ori` loads 7 and `bcast` copies it into every PE. Using `$iproc` as an input then gives each PE a different result, so a wrong PE stands out.
- Each instruction writes its own register (`$2`–`$19`), and some results feed later instructions: `psltiu` makes the flag `offeq` uses, and `pxori` pairs each PE with a partner (PE 0 with 1, 2 with 3) so `net` can swap values between them.
- The enable instructions write no register, so each is followed by a `paddiu` with the PEs receiving it showing up as enabled. A nested `pushen`/`popen` pair shows that each `popen` removes only one level.
- The `por` and `pori` operands make OR and ADD give different results. `gor` gives the same answer for both with 2 PEs, so it was also checked with 4 PEs (OR gives `f`, ADD would give 22).

AIK's output for the program matches the testbench exactly. With 1, 2, 4, and 8 PEs, every write matched the expected value and the program halted cleanly.

2 Issues

2.1 Specification Changes

AIK 20241001 filled the unused fields of `pushen` and `popen` with nonzero values that varied, so the same instruction could assemble to different words. We changed those fields to zeros in our specification, so their encodings are now always `0x40000004` and `0x40000005`.

2.2 Limitations

The enable interlock is conservative meaning the later SIMD instructions wait until an `offeq`, `pushen`, or `popen` has finished, trading some speed for simple correct timing while CU instructions can continue. The processor has no exception unit, so memory alignment and bounds are up to the program.